

- RAG Learning

3.2 Programming

AQA GCSE Computer Science (8525) — Paper 1

3.2.1 Data Types

A **data type** is a classification that tells the computer what kind of value a variable holds, how much memory to allocate for it, and what operations can legally be performed on it. Choosing the correct data type is a fundamental part of writing a reliable program — using the wrong type causes bugs, wastes memory, or produces unexpected results (for example, concatenating instead of adding).

Every value a program handles — a price, a name, a password, a temperature reading, a win/lose flag — must have a data type. The AQA specification requires you to know **five basic types**: integer, real (float), Boolean, character and string. You should be able to recognise each in pseudo-code, choose the appropriate type for a given scenario, and convert (cast) between them when needed.

The Five AQA Data Types

Type	What it stores	Example values	Typical use
Integer	A whole number, positive or negative, with no fractional part	42 , 0 , -7 , 1000000	Counting (lives left, stock level, age in years, loop counters)
Real / Float	A number with a decimal point (fractional part)	3.14 , -0.5 , 99.99 , 0.0	Measurements, prices, percentages, scientific values

Boolean	One of only two values: True or False	True , False	Flags and conditions (is_logged_in, game_over, has_paid)
Character	A single symbol from the character set (letter, digit, punctuation, space)	'A' , '7' , '?' , ' '	Single-letter menu choice, chess piece code, initial of a name
String	A sequence of zero or more characters, treated as a single value	"Hello" , "Z4X 9PQ" , ""	Names, addresses, passwords, messages

Note that a **character is not the same as a string**, even when the string contains only one letter. 'A' (in single quotes) is a character; "A" (in double quotes, with length 1) is a string. Similarly, the digit '7' as a character is not the same as the integer 7 ; the character holds the *symbol* 7, not the *quantity* 7, and they have different binary representations (the character uses its ASCII / Unicode code point).

Exam Tip: When asked "which data type is most appropriate for X?" choose the *smallest* or *most specific* type that can hold the value. A pupil's age is an integer, not a real (you do not need 12.47 years). A question answered yes/no is a Boolean, not a string "yes"/"no". A postcode is a string, not an integer (it contains letters). Marks are lost for storing numeric data as strings, as this blocks arithmetic without casting.

Casting (Type Conversion)

Casting (or **type conversion**) is converting a value from one data type to another. It is needed when a value arrives in one form (e.g. text from keyboard input) but must be used in another (e.g. arithmetic on a number). AQA pseudo-code uses functions named after the destination type: `int()` , `float()` (or `real()`) , `str()` , `bool()` .

The most common place casting appears is reading input from the user. Keyboard input always arrives as a **string**, even if the user typed digits. If you want to do arithmetic you must cast it first.

Forgetting to do so is one of the most common beginner bugs and is frequently tested in AQA exam questions.

```
# Without casting – BUG
age ← USERINPUT          # age is the string "16"
next_age ← age + 1        # ERROR or "161" depending on language

# With casting – correct
age ← int(USERINPUT)      # age is the integer 16
next_age ← age + 1        # next_age is 17

# Casting for output
price ← 9.99
OUTPUT "Total: £" + str(price) # concatenation requires a string
```

Cast	Example	Notes
<code>int(x)</code>	<code>int("42")</code> → 42	String of digits → integer. <code>int("3.9")</code> may error (not integer text). <code>int(3.9)</code> → 3 (truncates, does not round).
<code>float(x)</code>	<code>float("9.99")</code> → 9.99	String containing digits and optional decimal point → real number.
<code>str(x)</code>	<code>str(42)</code> → "42"	Any value → its textual form. Essential before concatenation.
<code>bool(x)</code>	<code>bool(0)</code> → False	Converts a value to True/False according to language rules (e.g. non-zero numbers are True, empty string is False).

3.2.2 Programming Concepts

Computers follow instructions in a strictly ordered way. AQA requires you to understand the three core building blocks of every algorithm — **sequence**, **selection** and **iteration** — plus the ideas of **variables**, **constants**, **assignment**, and **meaningful identifiers**.

Variables and Constants

A **variable** is a named memory location that holds a value which can change while the program runs. A **constant** is a named memory location that holds a value which cannot change after it is first set. Both have a name (identifier) and a current value; the only difference is whether the value is allowed to change.

Think of a variable as a labelled box. The label (`score` , `user_name`) never changes, but the value inside the box can be replaced as the program runs. A constant is a labelled box that has been **locked shut** after you put its value in — you (and future readers of the code) can still *read* the value but you cannot change it. Constants are used for values that should not change, such as mathematical constants (`PI = 3.14159`), configuration values (`VAT_RATE = 0.20` , `MAX_SCORE = 100`) or conversion factors.

```
# Variables (changeable)
score ← 0
score ← score + 10      # now 10
score ← score + 5       # now 15

# Constants (unchangeable, usually written in UPPERCASE by convention)
CONSTANT VAT_RATE = 0.20
CONSTANT MAX_LIVES = 3
price_with_vat ← price * (1 + VAT_RATE)
```

Using constants instead of "magic numbers" has three benefits: (1) the code is more **readable** because `VAT_RATE` explains what `0.20` means; (2) it is easier to **update** because you only change the value in one place; (3) it prevents accidental changes that would cause hard-to-find bugs.

Assignment

Assignment stores a value in a variable. AQA pseudo-code uses the left-arrow `←` (read as "becomes" or "is assigned"). Many textbooks and most programming languages use `=` or `:=`. The important thing is the **direction**: the value on the right is computed first, then stored in the variable on the left.

```
x ← 5           # x becomes 5
x ← x + 1       # right side first: (5 + 1) = 6, then stored in x. x is now 6.
name ← "Ada"    # name is a string
ready ← True    # ready is a Boolean
```

`x ← x + 1` is not an equation — it would be nonsense in mathematics. It means: take the current value of `x`, add 1 to it, and put the result back into `x`. This pattern is the basis of counters, accumulators and almost every loop.

Meaningful Identifiers

An **identifier** is the name given to a variable, constant, subroutine or other program element. A **meaningful identifier** clearly describes what the item represents. Good identifiers make code self-documenting — a reader can understand the purpose of a variable from its name alone, without needing comments.

Poor identifier	Better identifier	Reason
<code>x</code>	<code>total_price</code>	Explains what it stores
<code>n</code>	<code>number_of_pupils</code>	Explains the units
<code>flag</code>	<code>is_game_over</code>	Boolean meaning is explicit
<code>tmp</code>	<code>swap_buffer</code>	Purpose is clear
<code>dta</code>	<code>customer_records</code>	No cryptic abbreviations

Conventional rules: start with a letter, contain only letters, digits and underscores (no spaces), keep them short but descriptive, and use a consistent style (e.g. `snake_case` or `camelCase`). Loop counters (`i`, `j`, `k`) are the one accepted exception where single letters are still considered meaningful.

The Three Programming Constructs

Every algorithm can be built from just three types of flow of control. Being fluent in all three is central to writing correct programs.

1. Sequence

Sequence is the most basic: statements are executed *one after another, from top to bottom*, in the order they are written. Every piece of code is a sequence unless selection or iteration changes the flow.

```
OUTPUT "Welcome"
name ← USERINPUT
OUTPUT "Hello " + name
OUTPUT "Goodbye"
```

2. Selection (IF / ELSE IF / ELSE)

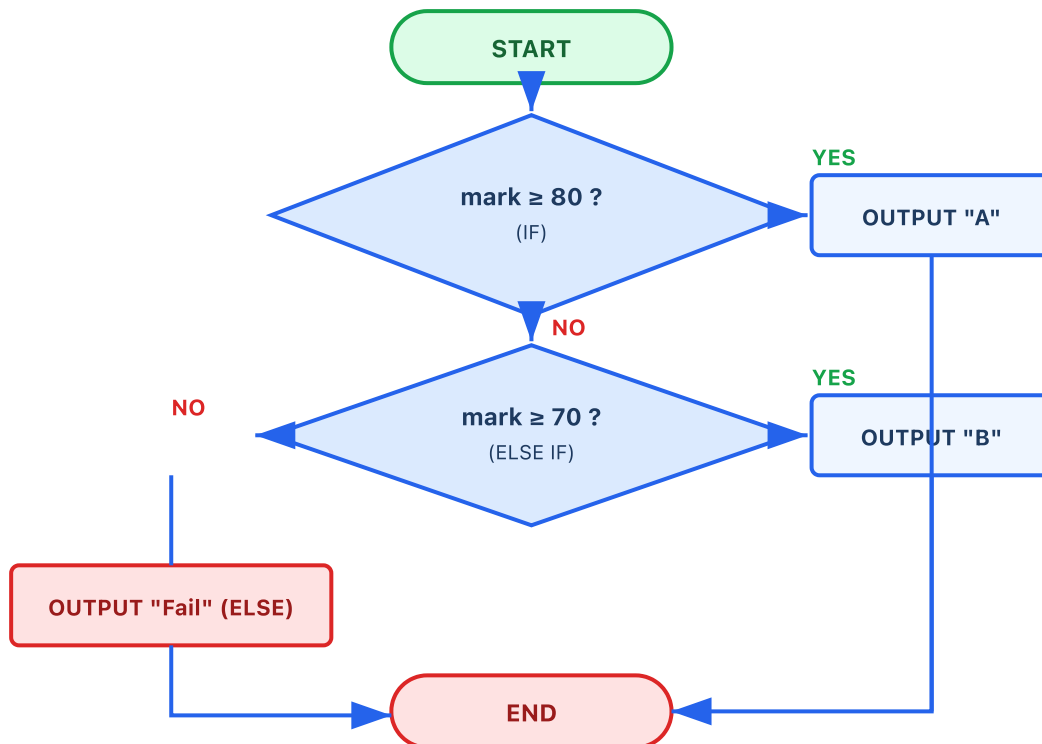
Selection means choosing which block of code to execute based on a condition. AQA uses `IF condition THEN ... ELSE IF condition THEN ... ELSE ... ENDIF`. Only one branch is executed each time the statement is reached — whichever one has the first condition evaluating to True (or the `ELSE` branch if none do).

```
INPUT mark
IF mark ≥ 80 THEN
    OUTPUT "A"
ELSE IF mark ≥ 70 THEN
    OUTPUT "B"
ELSE IF mark ≥ 60 THEN
    OUTPUT "C"
ELSE
    OUTPUT "Fail"
ENDIF
```

Note that the branches are **mutually exclusive**: for `mark = 85` only the first branch runs even though 85 is also ≥ 70 and ≥ 60 . This is why the order of `ELSE IF` tests matters — the most specific / highest condition must come first.

Selection: IF / ELSE IF / ELSE Flow of Control

Only one branch runs. The first True condition wins; ELSE runs if none are True.



3. Iteration (Loops)

Iteration means repeating a block of code. AQA distinguishes two kinds: **count-controlled** loops (`FOR`) that repeat a *known number of times*, and **condition-controlled** loops (`WHILE`) that repeat *while a condition is True* (an unknown number of times).

Count-Controlled Loops (FOR)

A `FOR` loop uses a loop counter that starts at a known value, increases by a step (usually 1) each time, and stops when it passes a known end value. Use it when you know *in advance* how many iterations are required.

```

# Print the numbers 1 to 5
FOR i ← 1 TO 5
    OUTPUT i
ENDFOR

# Sum the first 10 whole numbers
total ← 0
FOR i ← 1 TO 10

```

```

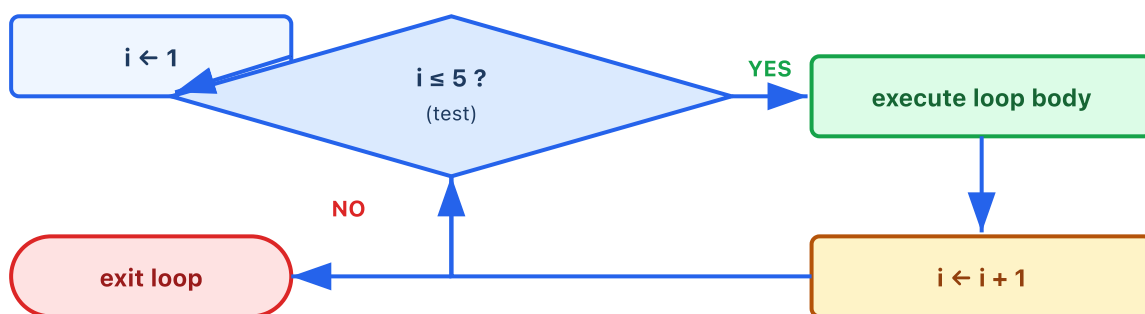
    total ← total + i
ENDFOR
OUTPUT total      # 55

# Step other than 1 (AQA uses T0 ... STEP)
FOR i ← 0 TO 20 STEP 2      # even numbers 0, 2, 4, ... 20
    OUTPUT i
ENDFOR

```

FOR Loop: FOR i ← 1 TO 5 ... ENDFOR

The loop counter i goes 1, 2, 3, 4, 5. After 5 the condition fails and the loop exits.



Trace (OUTPUT i each pass):

Pass 1 → i = 1 → OUTPUT 1
 Pass 2 → i = 2 → OUTPUT 2
 Pass 3 → i = 3 → OUTPUT 3

Pass 4 → i = 4 → OUTPUT 4
 Pass 5 → i = 5 → OUTPUT 5
 Pass 6 → i = 6 → test fails, exit

Condition-Controlled Loops (WHILE)

A **WHILE** loop keeps repeating while a condition is True. It is checked *before* each iteration, so the loop body might not run at all if the condition starts as False. Use it when you do not know in advance how many iterations are needed — for example, keep asking the user for a valid password, keep processing file lines until the end, keep applying interest until a balance exceeds a target.

```

# Keep asking until the user enters the correct password
password ← ""
WHILE password ≠ "hello"
    OUTPUT "Enter password: "
    password ← USERINPUT
ENDWHILE
OUTPUT "Access granted"

```

Worked Example — Trace Table for a FOR Loop

```
total ← 0
FOR i ← 1 TO 4
    total ← total + (i * i)
ENDFOR
OUTPUT total
```

Step	i	i * i	total	OUTPUT
initialise	—	—	0	
FOR start	1	1	1	
next i	2	4	5	
next i	3	9	14	
next i	4	16	30	
OUTPUT				30

Exam Tip: Choose FOR when you know the number of iterations up-front (e.g. "process all 30 pupils" or "draw a 10×10 grid"). Choose WHILE when repetition depends on a condition that becomes known only as the program runs (e.g. input validation, reading until end-of-file, searching until a flag changes). Using the wrong loop type is a frequent exam mistake.

3.2.3 Arithmetic Operations

AQA requires you to know the full set of arithmetic operators, including the two that beginners often miss: **integer division (DIV)** and **modulus (MOD)**, plus **exponentiation**.

Operation	Symbol(s)	Example	Result	Notes
-----------	-----------	---------	--------	-------

Addition	+	<code>5 + 3</code>	<code>8</code>	Adds two numbers (or concatenates strings in some languages)
Subtraction	-	<code>10 - 4</code>	<code>6</code>	Can produce negative results
Multiplication	*	<code>6 * 7</code>	<code>42</code>	Always use <code>*</code> — never <code>x</code> or implicit multiplication like <code>2a</code>
Real Division	/	<code>10 / 4</code>	<code>2.5</code>	Returns a real number even when dividing integers
Integer Division	DIV	<code>10 DIV 4</code>	<code>2</code>	Discards the remainder — gives the <i>whole</i> number of times the divisor fits
Modulus	MOD	<code>10 MOD 4</code>	<code>2</code>	Gives the <i>remainder</i> after integer division
Exponentiation	<code>^</code> or <code>**</code>	<code>2 ^ 3</code>	<code>8</code>	"Raised to the power of". <code>2^10 = 1024</code>

DIV and MOD — Why They Matter

Together, `DIV` and `MOD` split a division into its whole-number part and its remainder, exactly as you were taught to do long division in primary school. `23 ÷ 5 = 4 remainder 3` → `23 DIV 5 = 4` and `23 MOD 5 = 3`.

These two operators have many uses:

- **Test for even/odd:** `n MOD 2 = 0` is true for even numbers, `= 1` for odd.
- **Test divisibility:** `n MOD 5 = 0` tests whether `n` is a multiple of 5.
- **Extract the last digit:** `n MOD 10` gives the last digit of `n` (e.g. `2468 MOD 10 = 8`).
- **Strip the last digit:** `n DIV 10` removes the last digit of `n` (e.g. `2468 DIV 10 = 246`).
- **Convert total seconds to minutes and seconds:** `total DIV 60` = minutes, `total MOD 60` = leftover seconds.
- **Wrap around a range:** `i MOD n` always gives a value 0 ... n-1, useful for circular array indices.

```
# Convert 215 seconds to minutes and seconds
seconds ← 215
mins ← seconds DIV 60      # 3
secs ← seconds MOD 60     # 35
OUTPUT mins + " min " + secs + " s"  # "3 min 35 s"
```

Operator Precedence (Order of Operations)

When an expression mixes several operators, the computer follows the same precedence rules as mathematics (BIDMAS / PEMDAS). Use brackets to force a different order or simply to make intent clear.

Priority	Operators	Meaning
1 (highest)	()	Brackets override everything else
2	^ or **	Exponentiation
3	*, / , DIV , MOD	Multiplication, division and modulus
4 (lowest)	+, -	Addition and subtraction

```
2 + 3 * 4      # 14 (multiplication first, then addition)
(2 + 3) * 4    # 20 (brackets first)
10 - 6 / 2     # 7  (division first: 6/2 = 3, then 10 - 3)
2 ^ 3 * 2      # 16 (exponent first: 8, then *2)
17 MOD 5 + 2   # 4  (MOD first: 17 MOD 5 = 2, then +2)
```

Exam Tip: If an expression confuses you, rewrite it with brackets. `a + b * c` is clearer written `a + (b * c)`. Examiners accept either form as long as the result is correct, but unbracketed expressions are a common source of lost marks when students misjudge precedence.

3.2.4 Relational Operations

A **relational (comparison) operator** compares two values and returns a Boolean — True or False. They are the building blocks of the conditions used in selection and iteration.

Operation	AQA Symbol	Also written	Example (True / False)
Equal to	<code>=</code>	<code>==</code> (in Python, Java, C)	<code>5 = 5</code> → True
Not equal to	<code>≠</code>	<code>!=</code> or <code><></code>	<code>5 ≠ 3</code> → True
Less than	<code><</code>	<code><</code>	<code>3 < 5</code> → True
Less than or equal to	<code>≤</code>	<code><=</code>	<code>5 ≤ 5</code> → True
Greater than	<code>></code>	<code>></code>	<code>3 > 5</code> → False
Greater than or equal to	<code>≥</code>	<code>>=</code>	<code>5 ≥ 3</code> → True

Relational operators also work on strings, comparing *lexicographically* (dictionary order based on character codes). In AQA pseudo-code `"apple" < "banana"` evaluates to True because "apple" comes before "banana" alphabetically. Character comparisons use the same principle but on single characters' Unicode / ASCII values.

```

IF age ≥ 18 THEN
    OUTPUT "Adult"
ENDIF

IF name ≠ "" THEN          # ≠ is not equal
    OUTPUT "Hello " + name
ENDIF

WHILE score < target
    score ← score + 1
ENDWHILE

```

3.2.5 Boolean Operations

A **Boolean operation** combines or modifies Boolean (True/False) values. AQA requires three: **AND**, **OR**, **NOT**. They are used to build complex conditions out of simple ones.

Operator	Description	Example	Result
AND	Both sides must be True for the whole expression to be True	<code>age ≥ 18 AND has_ID = True</code>	True only if both conditions hold
OR	At least one side must be True	<code>day = "Saturday" OR day = "Sunday"</code>	True if either is True
NOT	Reverses / negates a Boolean	<code>NOT is_logged_in</code>	True if is_logged_in is False

Truth Tables

A	B	A AND B	A OR B	NOT A
F	F	F	F	T
F	T	F	T	T
T	F	F	T	F
T	T	T	T	F

```
# Combined Boolean conditions
IF age ≥ 13 AND age ≤ 19 THEN
    OUTPUT "Teenager"
ENDIF

IF is_admin = True OR is_owner = True THEN
    OUTPUT "Access granted"
ENDIF

WHILE NOT finished
```

```
# ... do more work ...  
ENDWHILE
```

Boolean operators follow a precedence of their own: `NOT` is evaluated first, then `AND`, then `OR`. But as with arithmetic, it is much clearer to use brackets: `(A AND B) OR C` is unambiguous.

Exam Tip: A classic exam pitfall is writing something like `IF grade = "A" OR "B"`. This is wrong — `"B"` on its own is not a condition. It must be written as `IF grade = "A" OR grade = "B"`. Each side of an `AND` / `OR` must be a complete Boolean expression.

3.2.6 Data Structures — Arrays and Records

A variable holds a single value. To handle collections of related values — the 30 marks of a class, the 7 days' rainfall, the 9 squares of a noughts-and-crosses board — we need a **data structure**. AQA requires you to know **1D arrays**, **2D arrays**, and (in some materials) **records**.

One-Dimensional Arrays

A **one-dimensional array** is an ordered collection of items of the same type, stored under a single identifier. Each item is accessed by its **index** (position). AQA uses **zero-based indexing**: the first item is at index `0`, the second at index `1`, and so on. An array of length n has valid indices `0` to $n-1$.

```
# Declaring and using a 1D array  
scores ← [72, 85, 64, 90, 58]  
  
OUTPUT scores[0]           # 72  
OUTPUT scores[3]           # 90  
scores[2] ← 70             # change the third item  
  
# Iterating over an array  
FOR i ← 0 TO length(scores) - 1  
    OUTPUT scores[i]  
ENDFOR
```

In AQA pseudo-code, `length(list)` returns the number of items. The loop therefore goes from `0` to `length(scores) - 1` inclusive — one of the most common array-traversal patterns you will see in the exam.

Two-Dimensional Arrays

A **two-dimensional array** is an array of arrays — a grid of values organised into rows and columns. Items are accessed with **two** indices: row first, then column. 2D arrays are used for tables of data, game boards, spreadsheets, images, matrices — anything with a natural rectangular shape.

```
# 3 rows, 4 columns – class scores for 4 tests
scores ← [ [72, 85, 64, 90],
            [58, 77, 81, 66],
            [91, 84, 73, 88] ]

OUTPUT scores[0][0]      # 72 (row 0, col 0 – top-left)
OUTPUT scores[2][3]      # 88 (row 2, col 3 – bottom-right)
scores[1][2] ← 85        # change row 1, col 2

# Nested loop to read every element
FOR row ← 0 TO 2
    FOR col ← 0 TO 3
        OUTPUT scores[row][col]
    ENDFOR
ENDFOR
```

Two-Dimensional Array: scores[row][col]

A 3×4 grid. scores[row][col] — zero-based indices. scores[1][2] = 81 is highlighted.

	col 0	col 1	col 2	col 3
row 0	72	85	64	90
row 1	58	77	81	66
row 2	91	84	73	88

scores[1][2]
= 81

Remember:

- Row index comes

first, column index **second**. • Indexing starts at 0. A 3×4 array uses rows 0–2 and columns 0–3. • An out-of-range index (e.g. scores[3][0]) is a run-time error.

Records

A **record** is a data structure that groups together a *fixed* set of named fields, which may be of *different* types. For example a Student record might contain a name (string), date of birth (string), year_group (integer), and is_present (Boolean). Unlike arrays — which hold many items of the same type accessed by an index — records hold one item of each type accessed by its **field name**.

```
# A record representing a student
student ← {
  name: "Ada Lovelace",
  year: 10,
  form: "10B",
  is_present: True
}

OUTPUT student.name      # Ada Lovelace
OUTPUT student.year      # 10
student.is_present ← False
```

In AQA pseudo-code, a record field is accessed with a dot (`student.name`). Records are very useful for representing real-world "things" whose properties are well-defined: a Book (title, author, ISBN, year), a Transaction (date, amount, payee, category), a Flight (code, origin, destination, departure_time, gate). An array of records is a common way to represent a table of data — e.g. an array of 30 Student records for a whole class.

3.2.7 Input / Output and File Handling

Programs need to communicate with the outside world — reading input from the user or from a file, and writing output to the screen or a file. AQA 8525 requires two channels: **keyboard input** and **screen output**; **text file read/write** is covered below as useful practical knowledge.

Keyboard Input and Screen Output

```
OUTPUT "Please enter your name: "  
name ← USERINPUT          # reads a line typed by the user  
OUTPUT "Hello, " + name  
  
# Numeric input requires casting (USERINPUT returns a string)  
OUTPUT "Enter your age: "  
age ← int(USERINPUT)  
OUTPUT "Next year you will be " + str(age + 1)
```

`OUTPUT` displays text on the screen. `USERINPUT` (sometimes just `INPUT`) reads a line of text from the keyboard, *including anything the user types right up to when they press Enter*. The value returned is always a **string**, even if they typed digits, so numeric input must be cast before arithmetic.

Text File Handling

A **text file** stores data as a sequence of lines of characters. The four basic file operations are: **open**, **read** (a line), **write** (a line), and **close**.

Operation	AQA pseudo-code	Purpose
-----------	-----------------	---------

Open	<pre>myFile ← openRead("names.txt") myFile ← openWrite("names.txt")</pre>	Open the file for reading or writing and store a handle in a variable. If opened for writing, any existing contents are usually overwritten.
Read line	<pre>line ← myFile.readLine()</pre>	Read the next line (up to but not including the end-of-line) and advance the internal position.
Write line	<pre>myFile.writeLine("Hello")</pre>	Write a line of text followed by a newline.
End of file	<pre>myFile.endOfFile()</pre>	Returns True when the reading position has reached the end.
Close	<pre>myFile.close()</pre>	Finish with the file. Closes the stream, flushes pending writes, and releases the file.

```
# Read every line of a text file and display it
myFile ← openRead("scores.txt")
WHILE NOT myFile.endOfFile()
    line ← myFile.readLine()
    OUTPUT line
ENDWHILE
myFile.close()
```

```
# Write three lines to a new text file
myFile ← openWrite("greetings.txt")
myFile.writeLine("Hello")
myFile.writeLine("Bonjour")
myFile.writeLine("Guten Tag")
myFile.close()
```

Exam Tip: Three essentials in every file-handling exam answer: (1) open the file (specifying read or write), (2) process it line by line (usually inside a WHILE NOT endOfFile loop for reading), (3) **close the file** at the end. Forgetting to close is a common mark-loser; it can leave the file locked or data unwritten.

3.2.8 String Handling

Strings are one of the most common data types in real programs, so AQA requires you to know the standard string operations.

Length, Indexing, Substring, Concatenation, Case

Operation	Pseudo-code	Example	Result
Length (number of characters)	<code>length(s)</code> or <code>len(s)</code>	<code>length("Hello")</code>	5
Indexing (single character by position)	<code>s[i]</code>	<code>"Hello"[0]</code>	"H"
Substring (slice)	<code>subString(s, start, length)</code> or <code>s[a:b]</code>	<code>subString("Hello", 1, 3)</code>	"ell"
Concatenation	<code>s1 + s2</code>	<code>"Hello " + "world"</code>	"Hello world"
Upper case	<code>toUpper(s)</code>	<code>toUpper("hi")</code>	"HI"
Lower case	<code>toLower(s)</code>	<code>toLower("HI")</code>	"hi"
Convert string ↔ integer	<code>int(s)</code> , <code>str(n)</code>	<code>int("42") + 1</code>	43

As with arrays, **string indexing is zero-based**: the first character of "Hello" is at index 0, the last at index 4. Accessing an index that does not exist is a run-time error.

```

name ← "Alan Turing"
OUTPUT length(name)           # 11
OUTPUT name[0]                # "A"   (first character)
OUTPUT name[5]                # "T"   (the 'T' of Turing)

initial ← name[0]
surname ← subString(name, 5, 6) # "Turing" (start index 5, length 6)

```

```
OUTPUT initial + " " + surname      # "A Turing"

# Iterating over every character
FOR i ← 0 TO length(name) - 1
    OUTPUT name[i]
ENDFOR
```

String to Integer and Back

Because user input and file data arrive as strings, converting between strings and numbers is extremely common. Always cast as soon as the type is wrong for the operation you want to do.

```
price_str ← "9.99"
price ← float(price_str)      # 9.99 as a real number
total ← price * 3             # 29.97

# Going the other way – building output
mins ← 3
secs ← 35
OUTPUT str(mins) + " min " + str(secs) + " s"    # "3 min 35 s"
```

Exam Tip: Two strings that look the same but are typed differently are not equal. Watch out for trailing spaces ("hello " vs "hello"), different cases ("Hello" vs "hello"), and "7" versus the integer 7 . To compare ignoring case, convert both sides with toLower() first. To compare with any whitespace removed, use a trim() function.

3.2.9 Random Number Generation

A **random number** is a value chosen unpredictably from a specified range. Programs use random numbers to simulate chance events — dice rolls, card shuffles, lottery draws, procedurally-generated levels, AI behaviour — and to pick test data, encryption keys, or unique IDs. Truly random numbers are hard to produce by software alone, so most languages provide **pseudo-random** generators that behave randomly enough for most purposes.

AQA pseudo-code uses a `random(low, high)` function that returns an integer chosen uniformly from `low` up to and including (or up to but not including, depending on the spec) `high`. Check the exam rubric; this guide uses *inclusive both ends*.

```
# Simulate a single six-sided die roll
dice ← random(1, 6)
OUTPUT "You rolled " + str(dice)

# Pick a random letter of the alphabet
letter_code ← random(65, 90)      # ASCII codes for 'A' to 'Z'
letter ← CHR(letter_code)

# Choose a random element of an array
favourites ← ["Dog", "Cat", "Rabbit", "Fish", "Hamster"]
idx ← random(0, length(favourites) - 1)
OUTPUT "Today's pet of the day is: " + favourites[idx]
```

Exam Tip: A frequent exam question is: "Write code to simulate rolling a pair of dice." The answer is **two separate calls** to `random(1, 6)`, *not* one call to `random(2, 12)`. Calling once gives 2–12 with equal probability, which is wrong — 7 should be six times more likely than 2 on real dice. Two dice combined give a triangular distribution.

3.2.10 Subroutines: Procedures and Functions

A **subroutine** is a named block of code that performs a specific task. You can call (invoke) a subroutine by its name from elsewhere in your program, passing in any values it needs as **parameters**. AQA distinguishes two kinds: a **procedure** carries out a task (e.g. prints a menu, writes to a file) but does not return a value; a **function** returns a value to its caller.

Procedures

```
PROCEDURE greet(name)
    OUTPUT "Hello, " + name + "!"
    OUTPUT "Welcome to the program."
```

```
ENDPROCEDURE
```

```
# Calling the procedure
greet("Alice")           # prints "Hello, Alice!" then "Welcome to the program"
greet("Bob")             # prints "Hello, Bob!" then "Welcome to the program"
```

Functions

```
FUNCTION circle_area(radius)
    CONSTANT PI = 3.14159
    RETURN PI * radius * radius
ENDFUNCTION

area ← circle_area(5)           # area is now 78.53975
OUTPUT "Area = " + str(area)
OUTPUT "Area of r=10 is " + str(circle_area(10)) # functions can be used in expressions
```

A **RETURN** statement inside a function both produces the result *and* ends the function early, so code after a taken **RETURN** is not executed. Procedures usually do not have a **RETURN** and simply end when the last statement runs (or optionally end early with **RETURN** on its own with no value).

Parameters and Arguments

A **parameter** is a variable named in the subroutine definition (the placeholder). An **argument** is the actual value supplied when the subroutine is called. In `circle_area(radius)` above, `radius` is the parameter; in `circle_area(10)`, `10` is the argument.

Subroutines may have zero, one, or many parameters. Each parameter has a name and (implicitly) a type inferred from the argument passed in.

```
FUNCTION rectangle_area(width, height)
    RETURN width * height
ENDFUNCTION

OUTPUT rectangle_area(4, 6)      # 24
OUTPUT rectangle_area(10, 2.5)  # 25.0
```

Local and Global Variables

A **local variable** is declared inside a subroutine and exists only while that subroutine is running — it is not visible to code outside. A **global variable** is declared outside all subroutines and is visible throughout the whole program. Using local variables where possible is strongly preferred because it keeps each subroutine **self-contained** and avoids unintended interactions.

```
GLOBAL high_score ← 0          # global – visible everywhere

PROCEDURE play_round()
    round_score ← random(1, 100)    # local to play_round
    IF round_score > high_score THEN
        high_score ← round_score    # we may update the global
    ENDIF
    OUTPUT "You scored " + str(round_score)
ENDPROCEDURE

play_round()
OUTPUT "Best so far: " + str(high_score)
# OUTPUT round_score          # ERROR – round_score is not visible here
```

Local variable	Global variable
Declared inside a subroutine	Declared outside all subroutines
Exists only while the subroutine is running	Exists for the whole program's life
Visible only inside that subroutine	Visible to every subroutine
Memory freed automatically when subroutine ends	Memory used for whole program run
Safer — cannot accidentally be changed elsewhere	Riskier — any subroutine can modify it
Preferred where possible	Use sparingly, e.g. for truly shared state (high_score, config)

Benefits of Subroutines

Subroutines are one of the most important tools in good programming. The benefits appear in almost every exam:

- **Reusability.** A subroutine is written once and can be called as many times as needed, with different arguments. Repeated code becomes a single definition.
- **Modularity.** A large program can be broken into small, focused subroutines — one job per subroutine. Each can be understood, tested, and replaced on its own.
- **Readability.** A line like `total ← calculate_bill(items, discount)` instantly communicates what is happening, even without reading the underlying code.
- **Maintainability.** When a rule changes (e.g. the VAT rate), you change it once in the subroutine instead of in many places throughout the program.
- **Easier debugging and testing.** Each subroutine can be tested with its own inputs, isolating bugs to smaller pieces of code.
- **Team working.** Different programmers can write different subroutines in parallel, as long as they agree on the parameters and return values.

Exam Tip: If a question asks "give three benefits of using subroutines", the safest answers are **reusability**, **readability**, and **easier to maintain / debug**. Always give *why* each helps, not just the word itself — e.g. "Subroutines make code reusable, because the same procedure can be called multiple times instead of copying the code each time."

3.2.11 Structured Programming

Structured programming is an approach to writing programs that uses only the three basic constructs (**sequence, selection, iteration**), avoids unrestricted jumps (like the classic `GOTO`), and decomposes programs into **self-contained subroutines** each solving a clear sub-problem. Structured programs have an obvious logical flow and are made of small, well-named blocks.

The term dates from the 1960s, when programs were often written as long tangles of `GOTO` statements jumping backwards and forwards — so-called "spaghetti code". Structured programming pioneers (Dijkstra, Wirth, Hoare) argued that almost all control flow can be expressed with sequence, selection, and iteration alone, and that doing so makes programs far easier to reason about, maintain, and debug. Today every mainstream programming language supports structured programming, and most of the AQA pseudo-code style is explicitly structured.

Advantages of Structured Programming

- **Easier to read and understand.** Anyone familiar with IF, FOR, WHILE and subroutine calls can follow the logic without tracing unexpected jumps.
- **Easier to maintain.** Each subroutine does one job, so changes are localised rather than rippling through the whole program.
- **Easier to test and debug.** Small, self-contained subroutines can be tested individually. When a bug appears, the area to search is small.
- **Promotes code reuse.** Well-designed subroutines can be called from many places, or even reused in future programs.
- **Supports teamwork.** Different team members can work on different subroutines independently as long as their parameters and return values are agreed.
- **Reliability.** Removing `GOTO` and other unconstrained jumps eliminates a whole class of subtle bugs.

Putting It All Together — A Worked Example

The program below reads a list of test scores from a file, calculates the average, and writes a short report. It combines almost everything in Topic 3.2: file handling, arrays, iteration, arithmetic (including `DIV` where appropriate), selection, string handling, casting, and subroutines with parameters and return values.

```
# ----- Subroutines -----  
  
FUNCTION read_scores(filename)  
    scores ← []  
    f ← openRead(filename)  
    WHILE NOT f.eofOfFile()  
        line ← f.readLine()
```

```
        scores.append(int(line))          # each line is one integer
    ENDWHILE
    f.close()
    RETURN scores
ENDFUNCTION

FUNCTION average(list)
    total ← 0
    FOR i ← 0 TO length(list) - 1
        total ← total + list[i]
    ENDFOR
    RETURN total / length(list)
ENDFUNCTION

PROCEDURE write_report(filename, avg)
    f ← openWrite(filename)
    IF avg ≥ 70 THEN
        f.writeLine("Class average: " + str(avg) + " - excellent")
    ELSE IF avg ≥ 50 THEN
        f.writeLine("Class average: " + str(avg) + " - satisfactory")
    ELSE
        f.writeLine("Class average: " + str(avg) + " - needs improvement")
    ENDIF
    f.close()
ENDPROCEDURE

# ----- Main program -----

scores ← read_scores("scores.txt")
avg     ← average(scores)
write_report("report.txt", avg)
OUTPUT "Report written. Average = " + str(avg)
```

Notice how the main program is only four lines long and reads almost like English. All the detail is hidden inside small, single-purpose subroutines — exactly the pattern structured programming encourages.

Exam Tip: When asked "state three advantages of structured programming", a strong answer pairs each advantage with a reason: "easier to debug because each subroutine can be tested on its own; easier to maintain because changes are localised to the relevant

subroutine; supports teamwork because different people can work on different subroutines in parallel." Avoid vague answers like "it is better" with no reason.